

Lecture 03

Memory-based Recommender Systems (part 3)

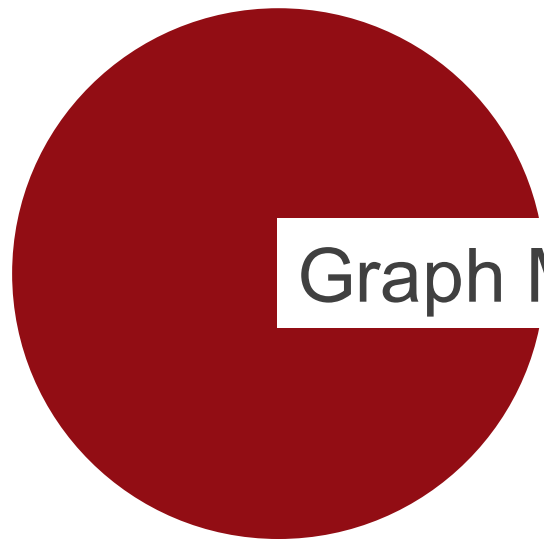
Euhyun Moon, Ph.D.
Machine Learning System (MLSys) Lab
Computer Science and Engineering
Sogang University

Slides adapted from J. Leskovec, A. Rajaraman, J. Ullman: Mining of Massive Datasets,
<http://www.mmids.org>, Stanford University



Today's Class

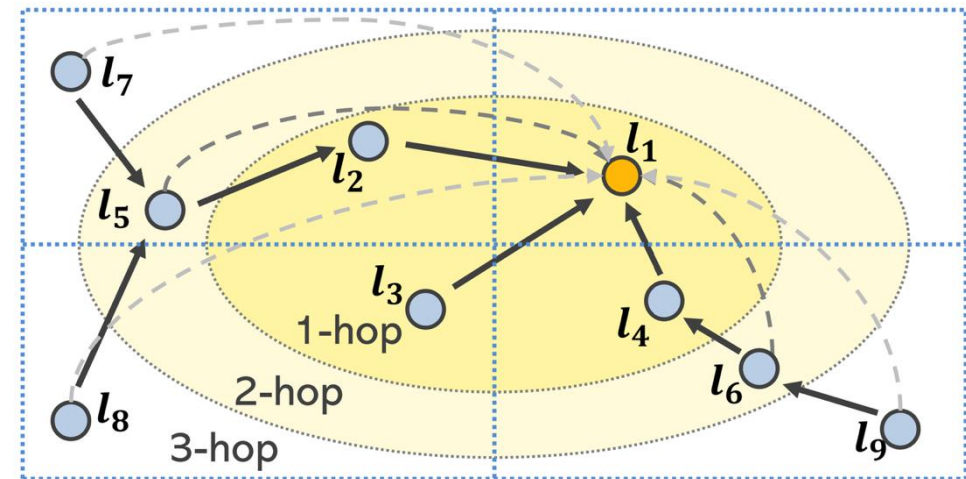
- Memory-based Recommender Systems
 - Graph models for neighborhood-based methods
 - Analysis of large graphs: PageRank



Graph Models for Neighborhood-based Methods

Introduction

- The sparsity of observed ratings causes a major problem in computing similarity for neighborhood-based methods
- Several **graph models** are used to define similarity in neighborhood-based methods, with the use of either structural transitivity or ranking techniques
- Graphs provide a structural representation of the relationships between users and/or items

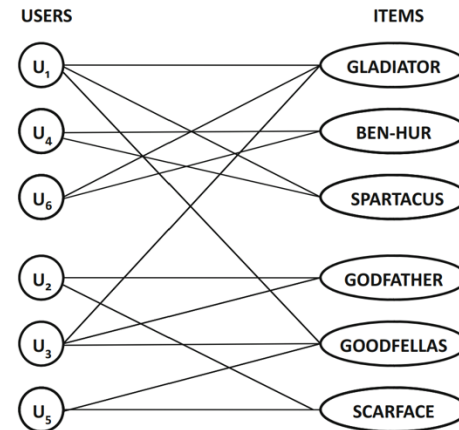


User-Item Graphs

- It is possible to use structural measures on the user-item graph, rather than Pearson correlation coefficient to define neighborhoods
- The user-item graph is defined as an undirected and bipartite graph $G = (N_u \cup N_i, A)$, where N_u represents the set of user nodes, and N_i represents the set of item nodes

	GLADIATOR	GODFATHER	BEN-HUR	GOODFELLAS	SCARFACE	SPARTACUS
U_1	1			5		2
U_2		5			4	
U_3	5	3		1		
U_4			3			4
U_5				3	5	
U_6	5		4			

(a) Ratings matrix



(b) User-item graphs of specified ratings

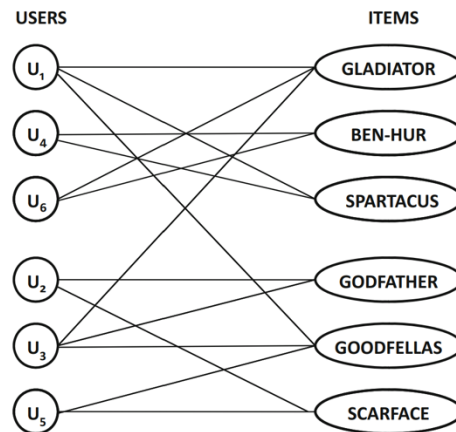
Advantages of User-Item Graphs

- The main advantage of graph-based methods is that two users can be considered neighbors even if they haven't rated many of the same items, as long as several short paths exist between them.
- This approach enables the construction of neighborhoods based on the concept of indirect connectivity between nodes
- Therefore, the graph-based approach provides an alternative method for defining neighborhoods, which is useful in sparse settings

Example - Representing Adjacency Matrix

	GLADIATOR	GODFATHER	BEN-HUR	GOODFELLAS	SCARFACE	SPARTACUS
U ₁	1			5		2
U ₂		5			4	
U ₃	5	3		1		
U ₄			3			4
U ₅				3	5	
U ₆	5		4			

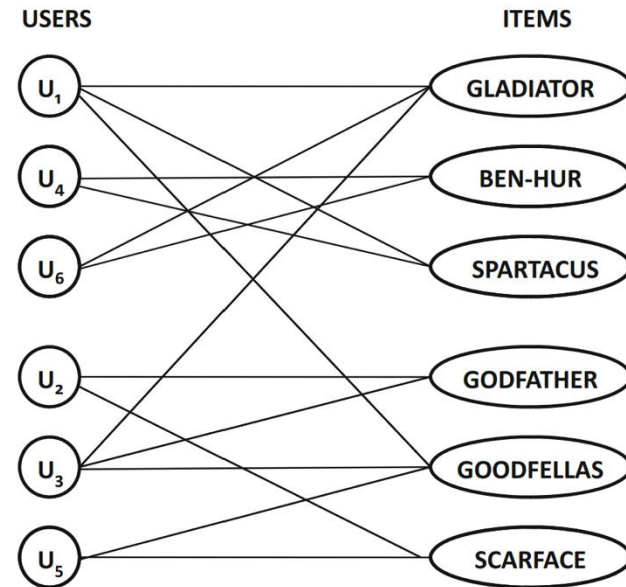
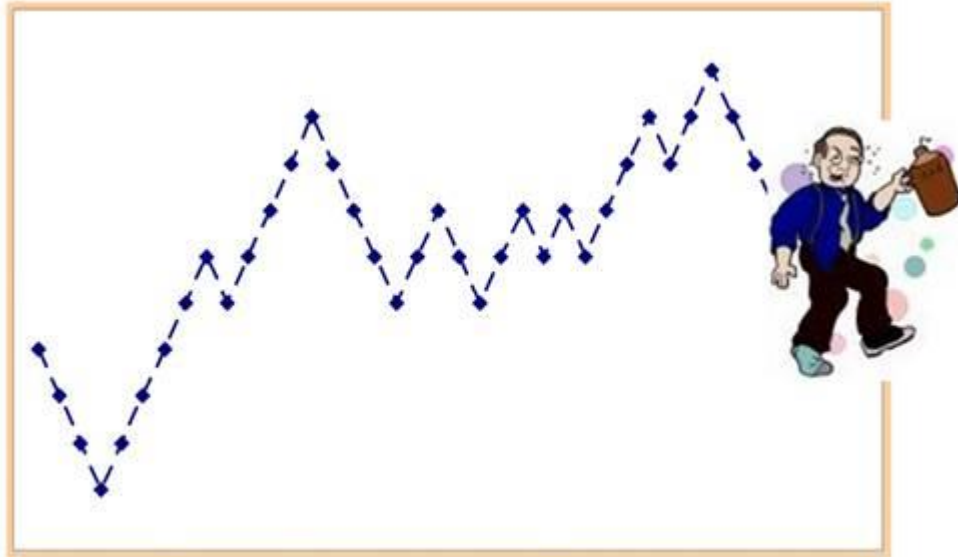
(a) Ratings matrix



(b) User-item graphs of specified ratings

Defining Neighborhoods with Random-Walk

- A **random-walk** method also considers indirect connectivity, because a walk from one node to another can take any number of steps



(b) User-item graphs of specified ratings

Defining Neighborhoods with Random-Walk

1) Construct the User-Item Graph:

- Create a graph in which users and items are represented as nodes

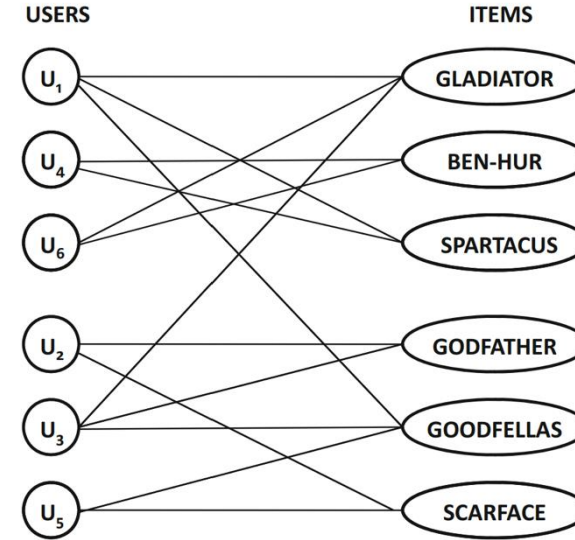
2) Define the Random Walk Process:

- Determine the parameters of the random walk, including the maximum number of steps, the probability of moving (transitioning) from users to items (and vice versa), and the stopping criterion

Defining Neighborhoods with Random-Walk

3) Perform Random Walks:

- Start random walks from each user node in the graph
- During the random walk, record the sequence of visited nodes, which typically includes both users and items
- The random walk can be stopped after a fixed number of steps or when a predefined stopping criterion is met



(b) User-item graphs of specified ratings

Defining Neighborhoods with Random-Walk

4) Calculate User Similarities:

- After generating random walks for all users, compute user-user similarities based on their walk sequences
- A common approach is to use Jaccard similarity, cosine similarity, or other similarity metrics to compare the sets of items visited by different users during their random walks
- For example, the Jaccard similarity between two users can be computed by comparing the sets of items they interacted with during their random walks

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}$$



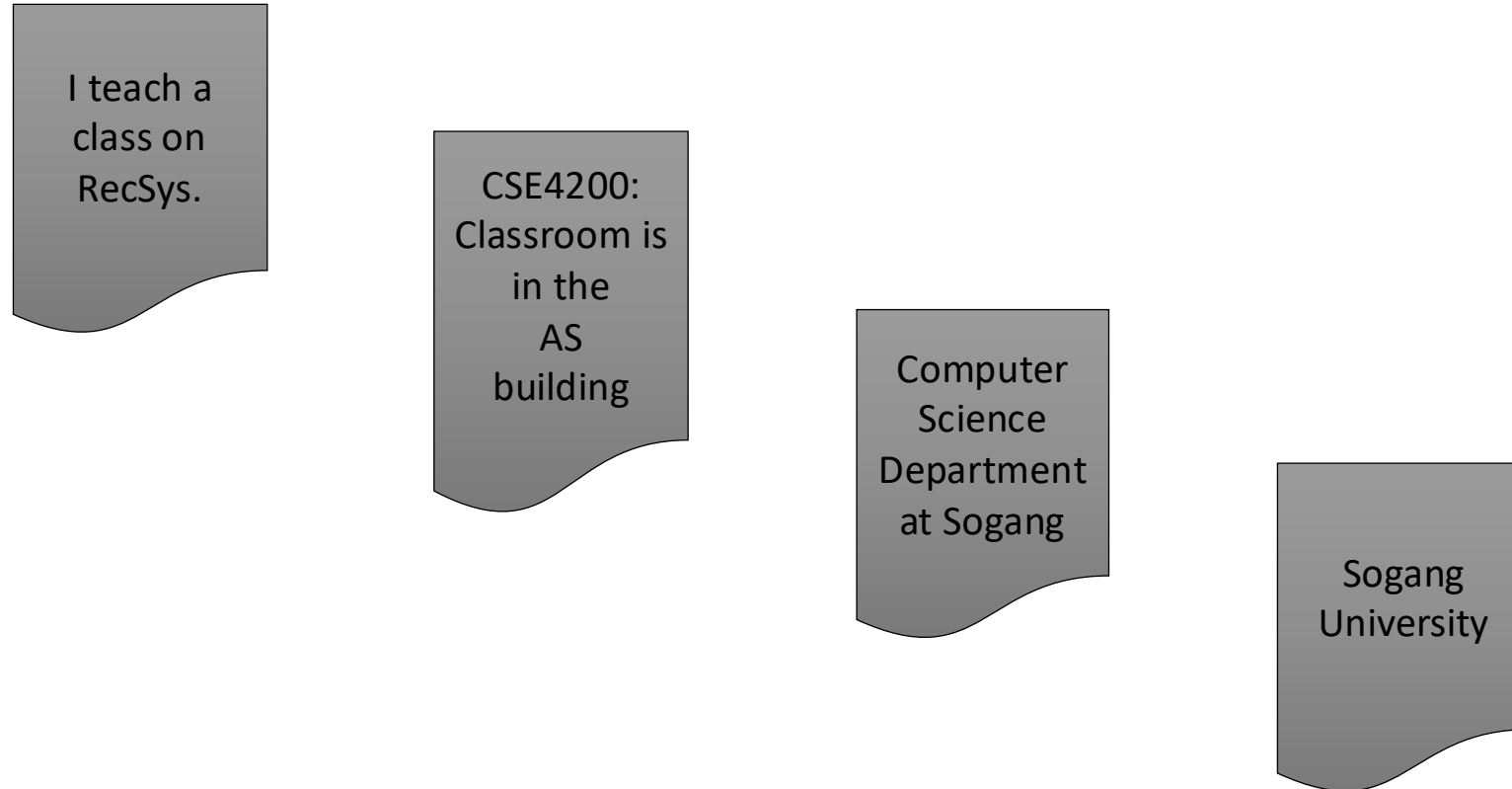
Analysis of Large Graphs: PageRank

PageRank



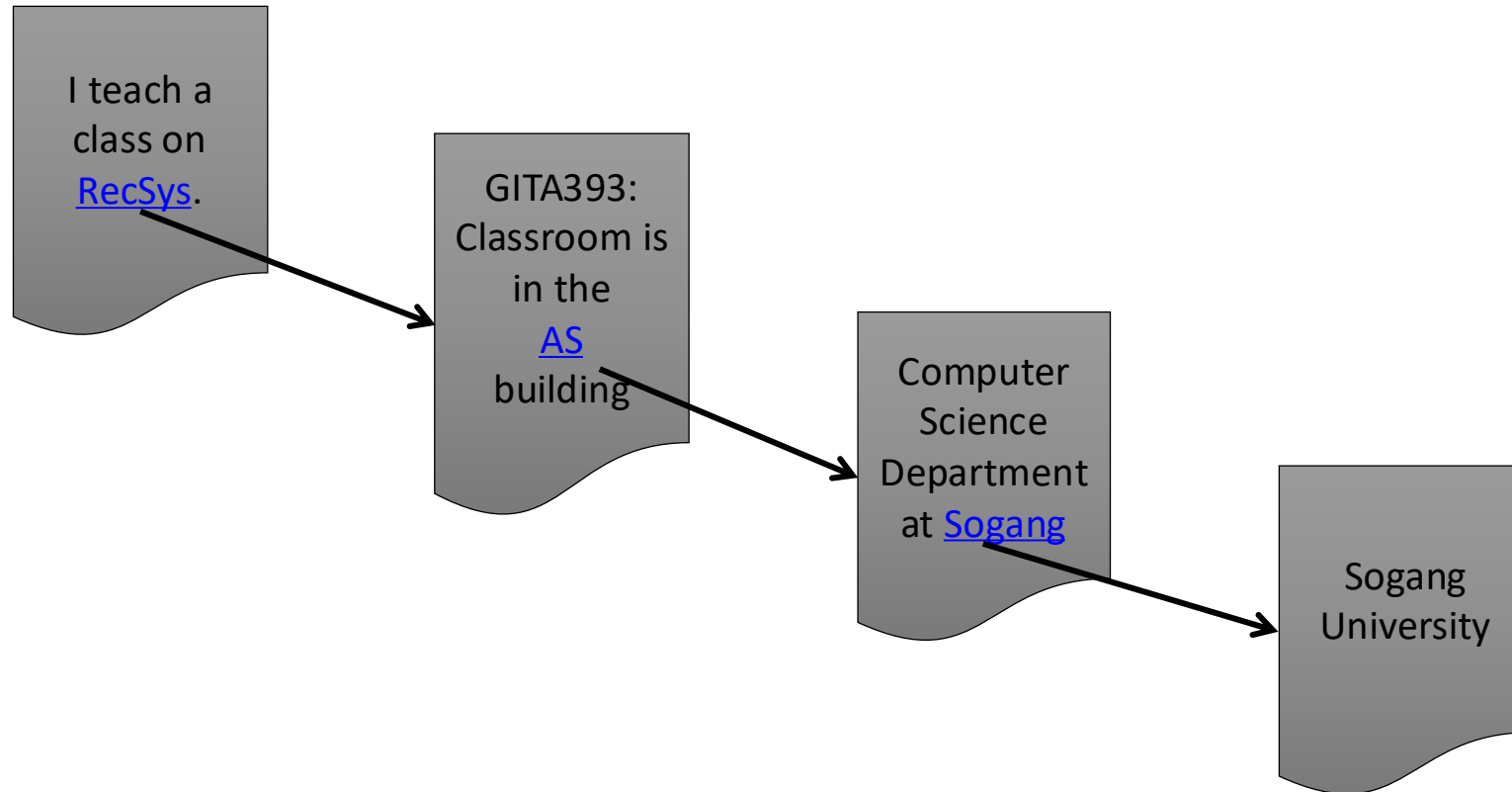
Web as a Graph

- **Web as a directed graph:**
 - **Nodes: Webpages**
 - **Edges: Hyperlinks**

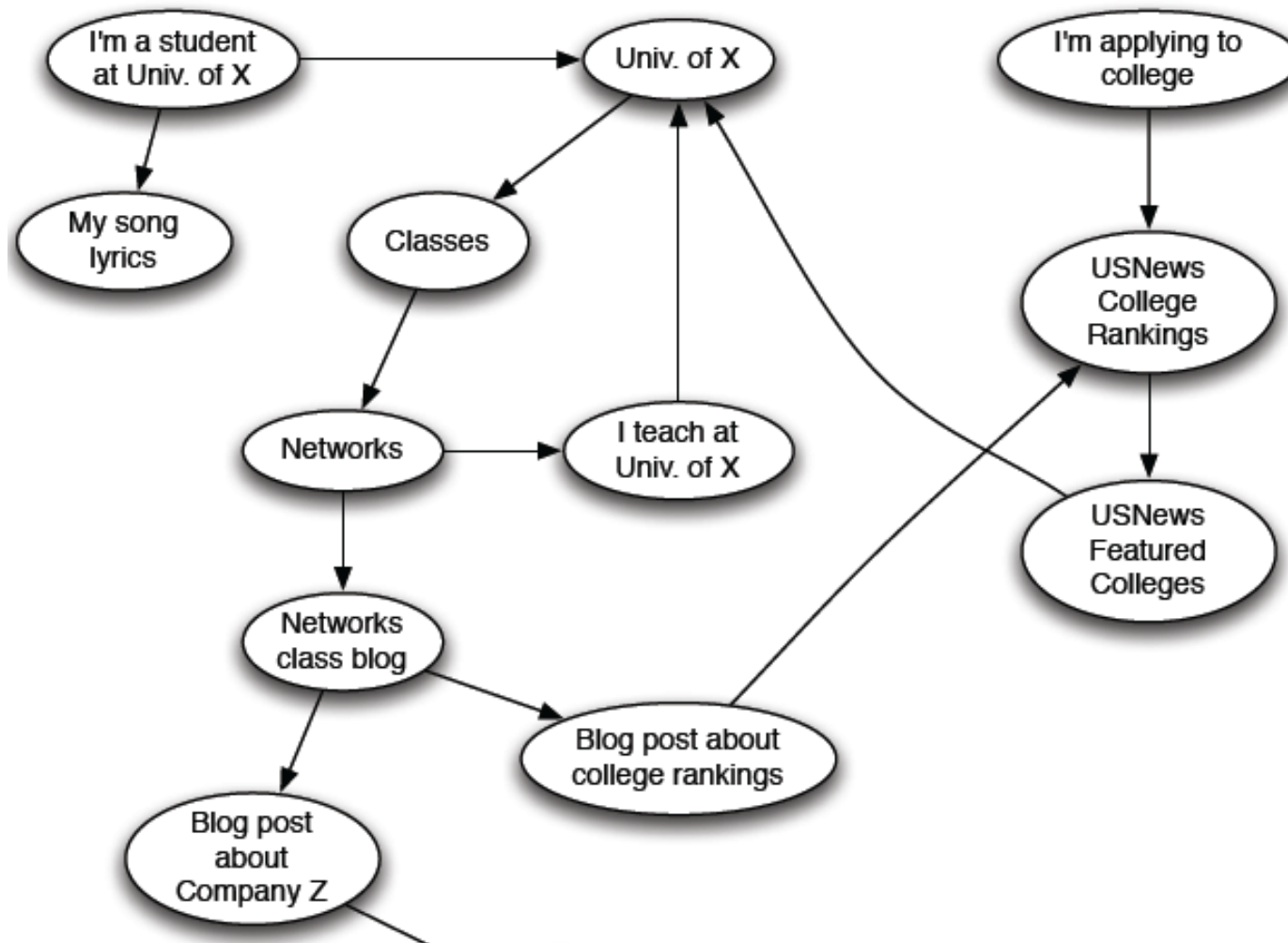


Web as a Graph

- **Web as a directed graph:**
 - **Nodes: Webpages**
 - **Edges: Hyperlinks**



Web as a Graph



Broad Question

- How to organize the Web?
- **First try:** Human curated **Web Directories**
 - Yahoo, DMOZ, LookSmart
- **Second try:** **Web Search**
 - **Information Retrieval** investigates: Find relevant docs in a small and trusted set
 - Newspaper articles, Patents, etc.
 - **But:** Web is **huge**, full of untrusted documents, random things, web spam, etc.



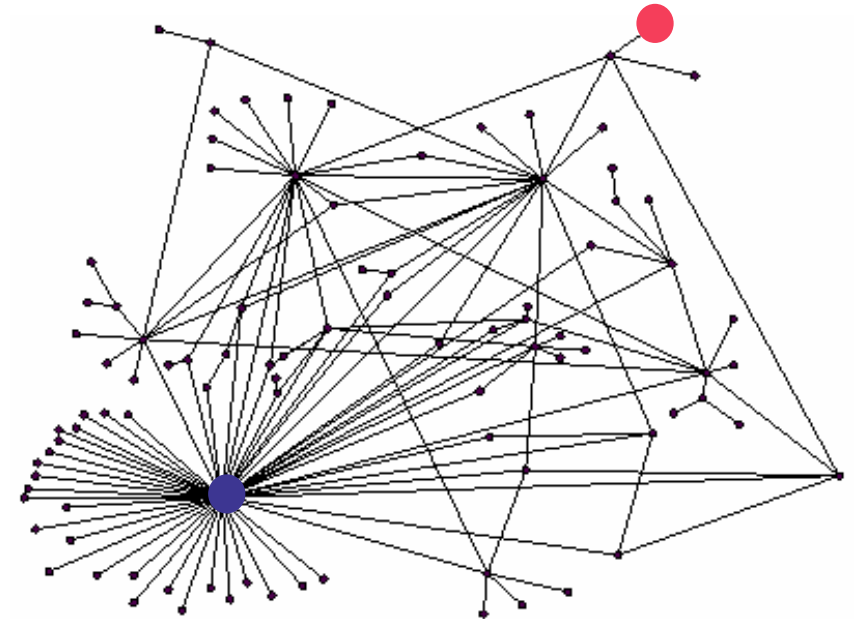
Web Search: Two Challenges

Two challenges of web search:

- (1) Web contains many sources of information
Who to “trust”?
 - **Trick:** Trustworthy pages may point to each other!
- (2) What is the “best” answer to query “newspaper”?
 - No single right answer
 - **Trick:** Pages that actually know about newspapers might all be pointing to many newspapers

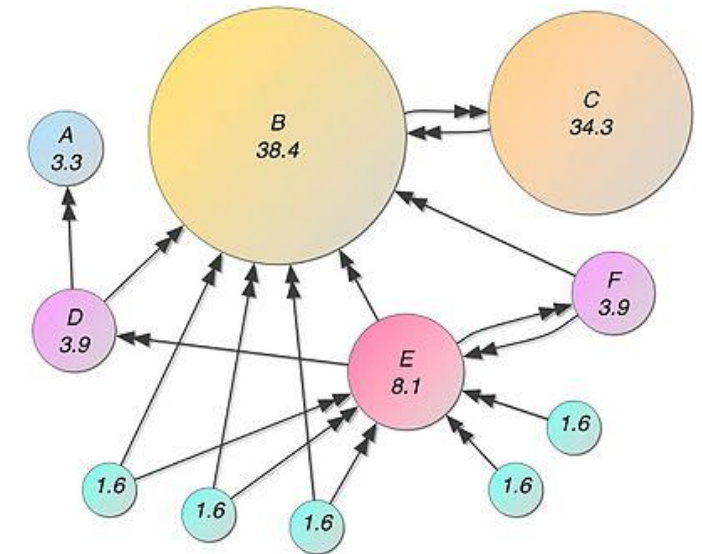
Ranking Nodes on the Graph

- All web pages are not equally “important”
<https://mlsys.sogang.ac.kr> vs. www.sogang.ac.kr
- There is large diversity in the web-graph node connectivity.
Let's rank the pages by the link structure!



PageRank

- The **PageRank** algorithm was initially proposed in the context of Web search, with the primary motivation of improving search quality
- One problem with the earliest search engines was that relying solely on content-centric matching of web pages with keywords for ranking often resulted in poor-quality results
- A mechanism was needed to determine the reputation and quality of web pages. This was achieved by leveraging the web's citation structure. When a page is of high quality, many other web pages link to it



Intuition behind PageRank – (1)

(1) Links as Votes

- **Idea: Links as votes**
 - Page is more important if it has more links
 - In-coming links? Out-going links?
- **Think of in-links as votes:**
 - www.sogang.ac.kr has 20,000 in-links
 - <https://mlsys.sogang.ac.kr> has 1 in-link
- **Are all in-links are equal?**
 - Links from important pages count more
 - Recursive question!

Intuition behind PageRank – (2)

(2) Random Surfing

- Web pages are important if people visit them a lot.
- But we can't watch everybody using the Web.
- A good surrogate for visiting pages is to assume people follow links randomly.
- Leads to *random surfer* model:
 - Start at a random page and follow random out-links repeatedly, from whatever page you are at.
 - *PageRank* = limiting probability of being at a page at any point in time.

Intuition behind PageRank – (3)

(3) Transition Matrix

- Solve the recursive equation:

“importance of a page = its share of the importance of each of its predecessor pages”

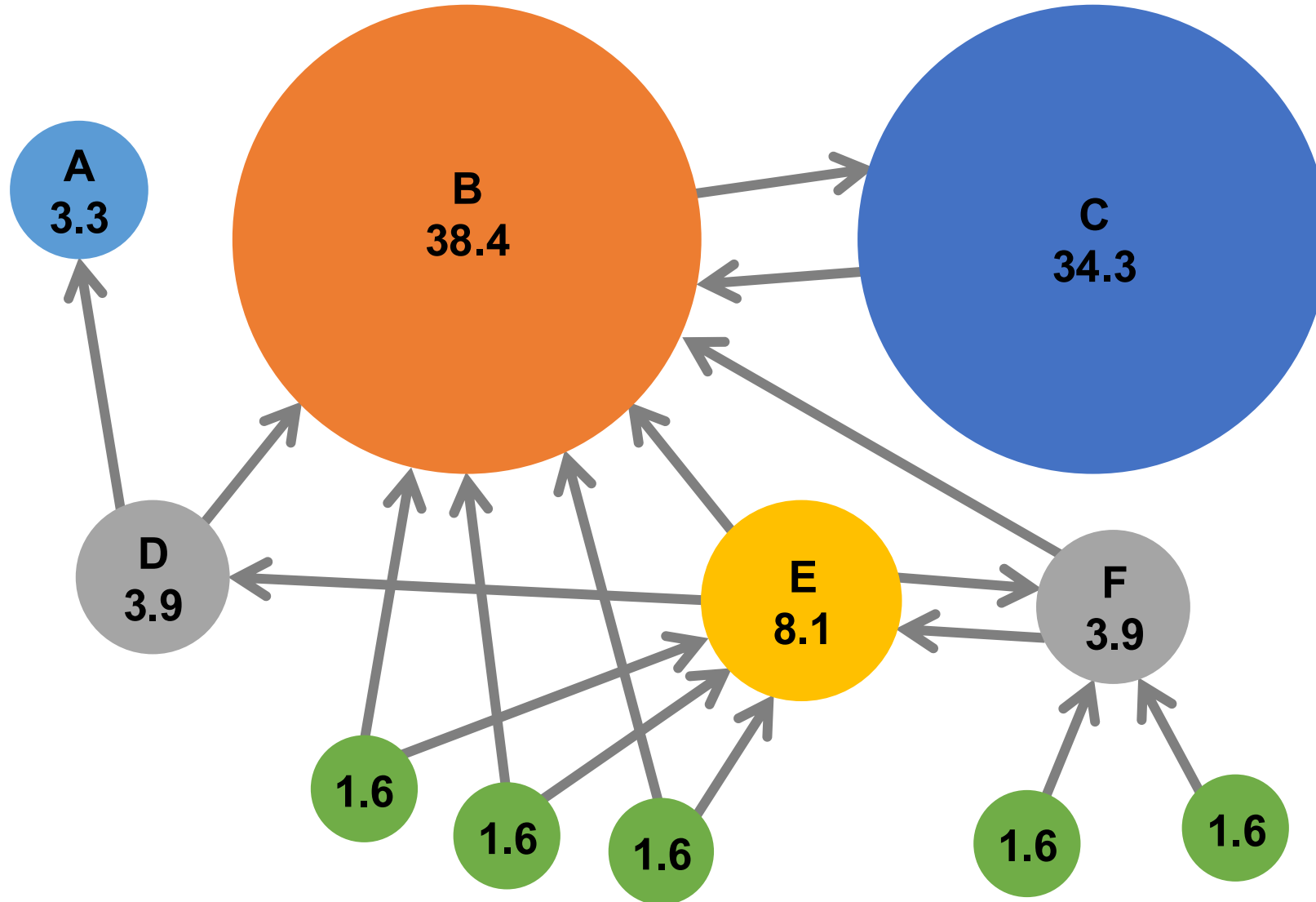
- Equivalent to the random-surfer definition of PageRank

- Technically,

importance = the principal eigenvector of the transition matrix of the Web

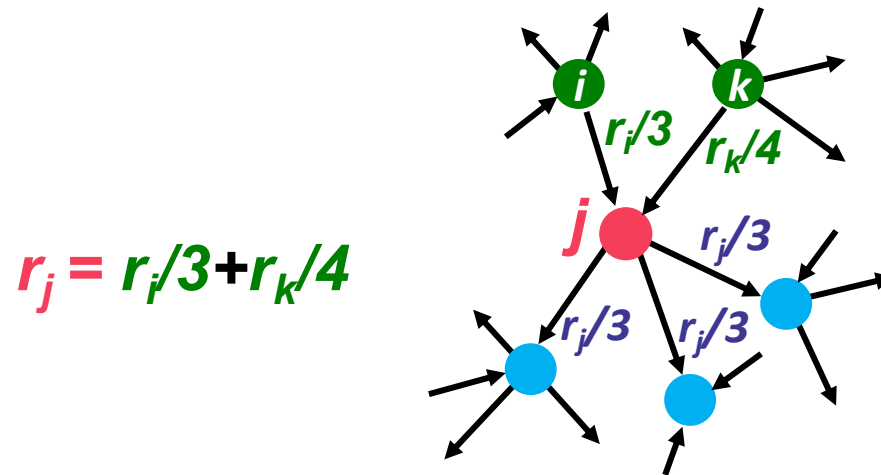
- A few fix-ups needed

Example: PageRank Scores



Simple Recursive Formulation

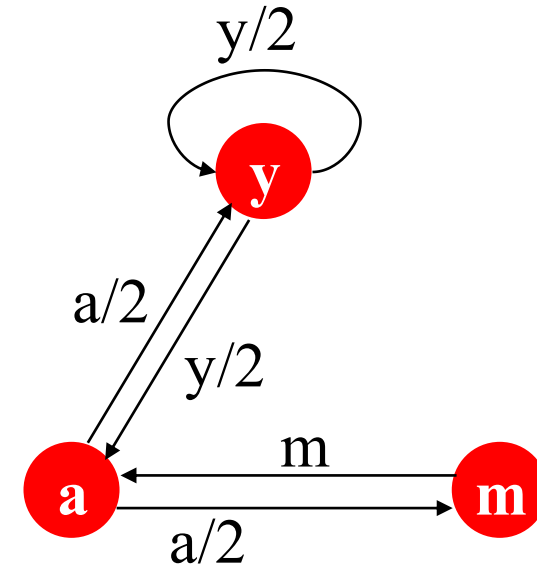
- Each link's vote is proportional to the **importance** of its source page
- If page j with importance r_j has n out-links, each link gets r_j / n votes
- Page j 's own importance is the sum of the votes on its in-links



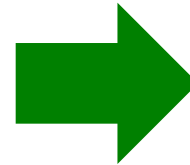
PageRank: The “Flow” Model

- A “vote” from an important page is worth more
- A page is important if it is pointed to by other important pages
- Define a “rank” r_j for page j

The web in 1839



$$r_j = \sum_{i \rightarrow j} \frac{r_i}{d_i}$$



“Flow” equations:

$$r_y = r_y/2 + r_a/2$$

$$r_a = r_y/2 + r_m$$

$$r_m = r_a/2$$

d_i ... out-degree of node i

r_j are the solutions to the “flow” equation

Solving the Flow Equations

- **3 equations, 3 unknowns, no constants**

- No unique solution
- All solutions equivalent modulo the scale factor

“Flow” equations:

$$r_y = r_y/2 + r_a/2$$

$$r_a = r_y/2 + r_m$$

$$r_m = r_a/2$$

- **Additional constraint forces uniqueness:**

- $r_y + r_a + r_m = 1$
- **Solution:** $r_y = 2/5, r_a = 2/5, r_m = 1/5$

- **Gaussian elimination method works for small examples, but we need a better method for large web-size graphs**

- **We need a new formulation!**

PageRank: Matrix Formulation

- **Define stochastic adjacency matrix M**
 - Let page i has d_i out-links
 - If $i \rightarrow j$, then $M_{ji} = \frac{1}{d_i}$ else $M_{ji} = 0$
 - M is a **column stochastic matrix**
 - Each column sums to 1
- **Define rank vector r :** a vector with an entry per page; it captures importance of the page
 - r_i is the importance score of page i
 - $\sum_i r_i = 1$
- **The flow equations can be written**

$$r = M \cdot r$$

$$r_j = \sum_{i \rightarrow j} \frac{r_i}{d_i}$$

Example

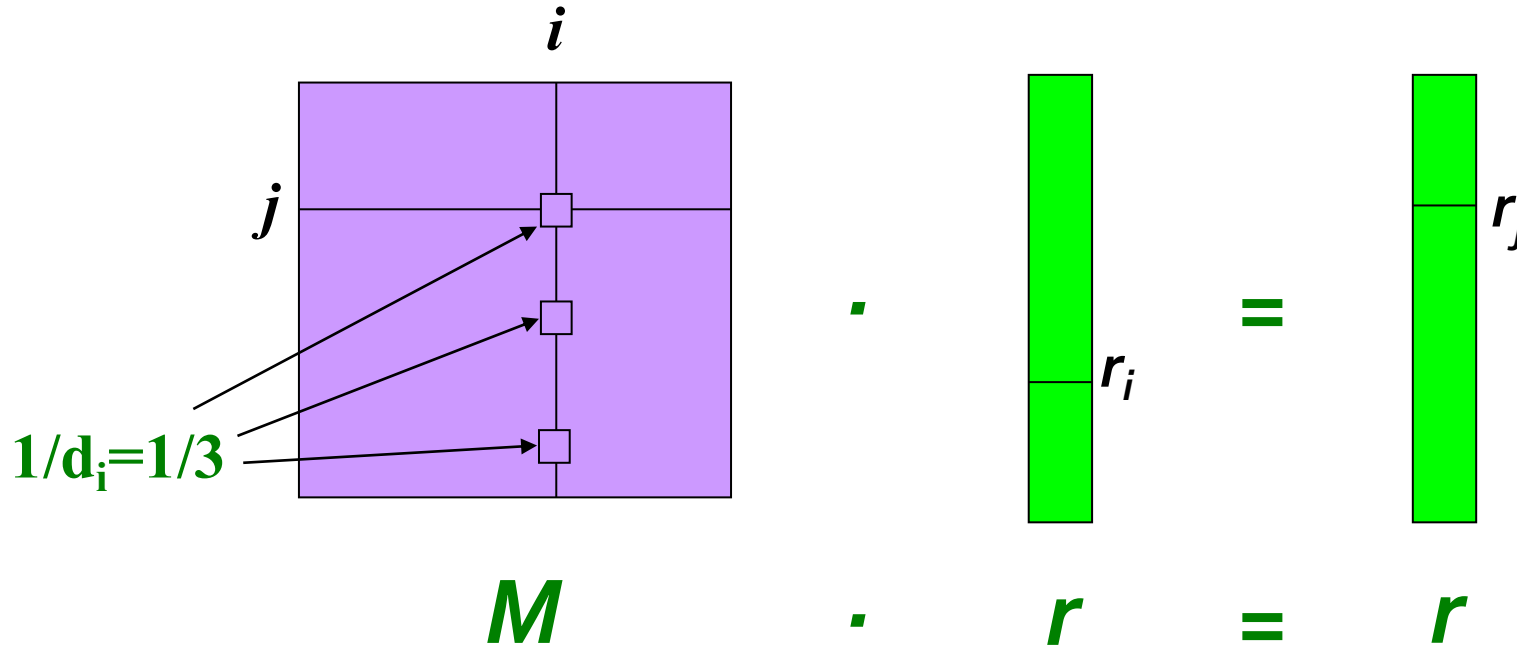
- Remember the flow equation:

$$r_j = \sum_{i \rightarrow j} \frac{r_i}{d_i}$$

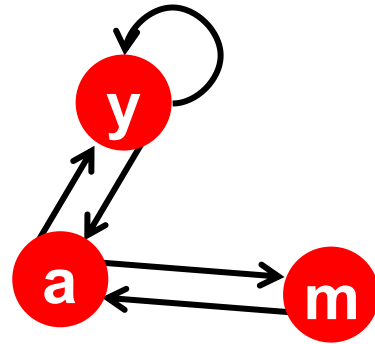
- Flow equation in the matrix form

$$M \cdot r = r$$

- Suppose page i links to 3 pages, including j



Example: Flow Equations & M



$$M = \begin{matrix} & \begin{matrix} y & a & m \end{matrix} \\ \begin{matrix} y \\ a \\ m \end{matrix} & \begin{bmatrix} 1/2 & 1/2 & 0 \\ 1/2 & 0 & 1 \\ 0 & 1/2 & 0 \end{bmatrix} \end{matrix}$$

$$\begin{aligned} r_y &= r_y/2 + r_a/2 \\ r_a &= r_y/2 + r_m \\ r_m &= r_a/2 \end{aligned}$$

$$\begin{bmatrix} r_y \\ r_a \\ r_m \end{bmatrix} = \begin{bmatrix} 1/2 & 1/2 & 0 \\ 1/2 & 0 & 1 \\ 0 & 1/2 & 0 \end{bmatrix} \cdot \begin{bmatrix} r_y \\ r_a \\ r_m \end{bmatrix}$$

Eigenvector Formulation

- The flow equations can be written

$$\mathbf{r} = \mathbf{M} \cdot \mathbf{r}$$

NOTE: $\mathbf{x}$ is an eigenvector with the corresponding eigenvalue λ if:

$$\mathbf{A}\mathbf{x} = \lambda\mathbf{x}$$

- So the **rank vector** $\mathbf{r}$ is an **eigenvector** of the stochastic web matrix $\mathbf{M}$
 - Starting from any stochastic vector $\mathbf{u}$, the limit $\mathbf{M}(\mathbf{M}(\dots\mathbf{M}(\mathbf{M}\mathbf{u})))$ is the long-term distribution of the surfers.
 - **The math:** limiting distribution = principal eigenvector of $\mathbf{M}$ = **PageRank**.
 - **Note:** If $\mathbf{r}$ is the limit of $\mathbf{M}\mathbf{M} \dots \mathbf{M}\mathbf{u}$, then $\mathbf{r}$ satisfies the equation $\mathbf{r} = \mathbf{M}\mathbf{r}$, so $\mathbf{r}$ is an eigenvector of $\mathbf{M}$ with eigenvalue 1
- **We can now efficiently solve for $\mathbf{r}$!**
The method is called Power iteration

Power Iteration Method

- Given a web graph with N nodes, where the nodes are pages and edges are hyperlinks
- **Power iteration:** a simple iterative scheme
 - Suppose there are N web pages
 - Initialize: $\mathbf{r}^{(0)} = [1/N, \dots, 1/N]^T$
 - Iterate: $\mathbf{r}^{(t+1)} = \mathbf{M} \cdot \mathbf{r}^{(t)}$
 - Stop when $|\mathbf{r}^{(t+1)} - \mathbf{r}^{(t)}|_1 < \varepsilon$

$$r_j^{(t+1)} = \sum_{i \rightarrow j} \frac{r_i^{(t)}}{d_i}$$

d_i out-degree of node i

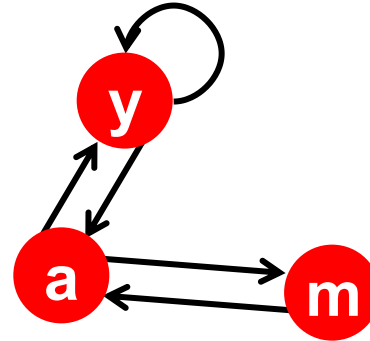
$|\mathbf{x}|_1 = \sum_{1 \leq i \leq N} |x_i|$ is the **L1** norm
So that $\mathbf{r}$ is a distribution (sums to 1)

About 50 iterations are sufficient to estimate the limiting solution.

PageRank: How to solve?

- **Power Iteration:**

- Set $r = [\frac{1}{N}, \frac{1}{N}, \frac{1}{N}]$
- 1: $r' = M \cdot r$
- 2: $r = r'$
- Goto 1



	y	a	m
y	$\frac{1}{2}$	$\frac{1}{2}$	0
a	$\frac{1}{2}$	0	1
m	0	$\frac{1}{2}$	0

$$r_y = r_y/2 + r_a/2$$

$$r_a = r_y/2 + r_m$$

$$r_m = r_a/2$$

- **Example:**

$$r = \begin{bmatrix} r_y \\ r_a \\ r_m \end{bmatrix} = \begin{bmatrix} 1/3 & 1/3 & 5/12 & 9/24 & & 6/15 \\ 1/3 & 3/6 & 1/3 & 11/24 & \dots & 6/15 \\ 1/3 & 1/6 & 3/12 & 1/6 & & 3/15 \end{bmatrix}$$

Iteration 0, 1, 2, ...

PageRank: The Google Formulation

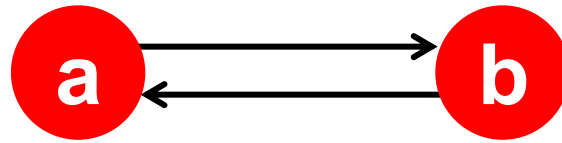
PageRank: Three Questions

$$r_j^{(t+1)} = \sum_{i \rightarrow j} \frac{r_i^{(t)}}{d_i} \quad \text{or equivalently} \quad r = Mr$$

- Does this converge?
- Does it converge to what we want?
- Are results reasonable?

Does this converge?

$$\mathbf{M} = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$



$$r_j^{(t+1)} = \sum_{i \rightarrow j} \frac{r_i^{(t)}}{d_i}$$

- **Example:**

$$\begin{pmatrix} r_a \\ r_b \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad \begin{pmatrix} 0 \\ 1 \end{pmatrix} \quad \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

Iteration 0, 1, 2, ...

Does it converge to what we want?

$$\mathbf{M} = \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix} \quad \text{a} \longrightarrow \text{b} \quad r_j^{(t+1)} = \sum_{i \rightarrow j} \frac{r_i^{(t)}}{d_i}$$

- **Example:**

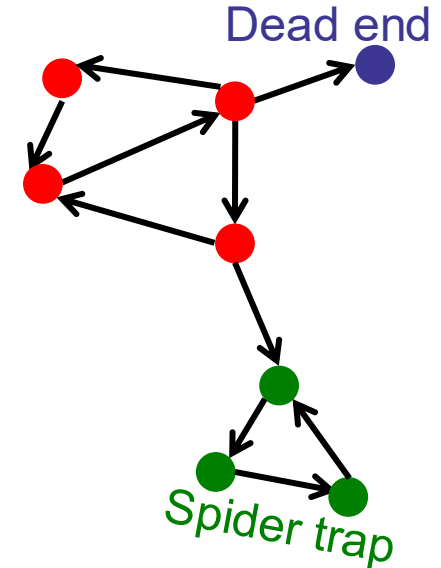
$$\begin{bmatrix} r_a \\ r_b \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad \begin{bmatrix} 0 \\ 1 \end{bmatrix} \quad \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Iteration 0, 1, 2, ...

PageRank: Problems

Two problems:

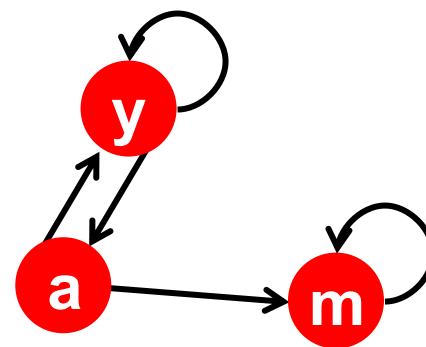
- (1) **Dead ends:** Some pages have no out-links
 - Random walk has “nowhere” to go to
 - Such pages cause importance to “leak out”
- (2) **Spider traps:**
(all out-links are within the group)
 - Random walk gets “stuck” in a trap
 - And eventually spider traps absorb all importance



Problem: Spider Traps

- **Power Iteration:**

- Set $r_j = 1/N$
- $r_j = \sum_{i \rightarrow j} \frac{r_i}{d_i}$
 - And iterate



m is a spider trap

	y	a	m
y	$\frac{1}{2}$	$\frac{1}{2}$	0
a	$\frac{1}{2}$	0	0
m	0	$\frac{1}{2}$	1

$$\mathbf{r}_y = \mathbf{r}_y / 2 + \mathbf{r}_a / 2$$

$$\mathbf{r}_a = \mathbf{r}_y / 2$$

$$\mathbf{r}_m = \mathbf{r}_a / 2 + \mathbf{r}_m$$

- **Example:**

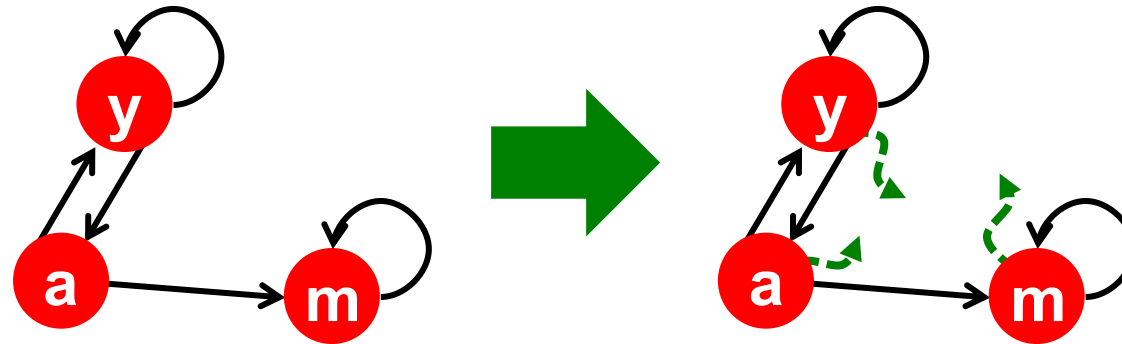
$$\begin{bmatrix} \mathbf{r}_y \\ \mathbf{r}_a \\ \mathbf{r}_m \end{bmatrix} = \begin{bmatrix} 1/3 & 2/6 & 3/12 & 5/24 & & 0 \\ 1/3 & 1/6 & 2/12 & 3/24 & \dots & 0 \\ 1/3 & 3/6 & 7/12 & 16/24 & & 1 \end{bmatrix}$$

Iteration 0, 1, 2, ...

All the PageRank score gets “trapped” in node m.

Solution: Probabilistically Teleport!

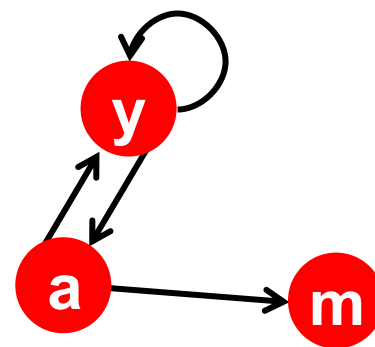
- **The Google solution for spider traps:** At each time step, the random surfer has two options
 - With prob. β , follow a link at random
 - With prob. $1-\beta$, jump to some random page
 - β is typically in the range 0.8 to 0.9
- **Surfer will teleport out of spider trap within a few time steps**



Problem: Dead Ends

- **Power Iteration:**

- Set $r_j = 1/N$
- $r_j = \sum_{i \rightarrow j} \frac{r_i}{d_i}$
 - And iterate



m is a dead end

	y	a	m
y	$\frac{1}{2}$	$\frac{1}{2}$	0
a	$\frac{1}{2}$	0	0
m	0	$\frac{1}{2}$	0

$$\mathbf{r}_y = \mathbf{r}_y / 2 + \mathbf{r}_a / 2$$

$$\mathbf{r}_a = \mathbf{r}_y / 2$$

$$\mathbf{r}_m = \mathbf{r}_a / 2 + \mathbf{r}_m$$

- **Example:**

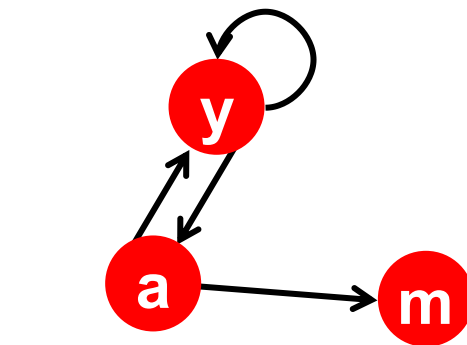
$$\begin{bmatrix} r_y \\ r_a \\ r_m \end{bmatrix} = \begin{bmatrix} 1/3 & 2/6 & 3/12 & 5/24 & & 0 \\ 1/3 & 1/6 & 2/12 & 3/24 & \dots & 0 \\ 1/3 & 1/6 & 1/12 & 2/24 & & 0 \end{bmatrix}$$

Iteration 0, 1, 2, ...

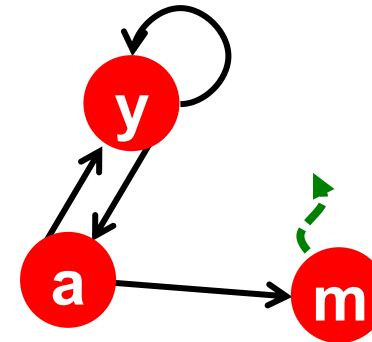
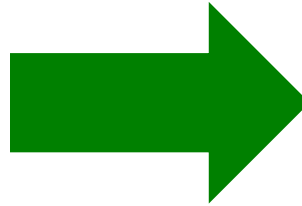
Here the PageRank score “leaks” out since the matrix is not stochastic.

Solution: Always Teleport!

- **Teleports:** Follow random teleport links with probability 1.0 from dead-ends
 - Adjust matrix accordingly



	y	a	m
y	$\frac{1}{2}$	$\frac{1}{2}$	0
a	$\frac{1}{2}$	0	0
m	0	$\frac{1}{2}$	0



	y	a	m
y	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{3}$
a	$\frac{1}{2}$	0	$\frac{1}{3}$
m	0	$\frac{1}{2}$	$\frac{1}{3}$

Why Teleports Solve the Problem?

Why are dead-ends and spider traps a problem
and **why do teleports solve the problem?**

- **Spider-traps** are not a problem, but with traps PageRank scores are **not** what we want
 - **Solution:** Never get stuck in a spider trap by teleporting out of it in a finite number of steps
- **Dead-ends** are a problem
 - The matrix is not column stochastic so our initial assumptions are not met
 - **Solution:** Make matrix column stochastic by always teleporting when there is nowhere else to go

Solution: Random Teleports

- Google's solution that does it all:

At each step, random surfer has two options:

- With probability β , follow a link at random
- With probability $1-\beta$, jump to some random page

- **PageRank equation** [Brin-Page, 98]

$$r_j = \sum_{i \rightarrow j} \beta \frac{r_i}{d_i} + (1 - \beta) \frac{1}{N}$$

d_i ... out-degree of node i

This formulation assumes that M has no dead ends.

We can either preprocess matrix M to remove all dead ends or explicitly follow random teleport links with probability 1.0 from dead-ends.

The Google Matrix

- **PageRank equation** [Brin-Page, '98]

$$r_j = \sum_{i \rightarrow j} \beta \frac{r_i}{d_i} + (1 - \beta) \frac{1}{N}$$

$[1/N]_{N \times N}$... N by N matrix
where all entries are $1/N$

- **The Google Matrix A :**

$$A = \beta M + (1 - \beta) \left[\frac{1}{N} \right]_{N \times N}$$

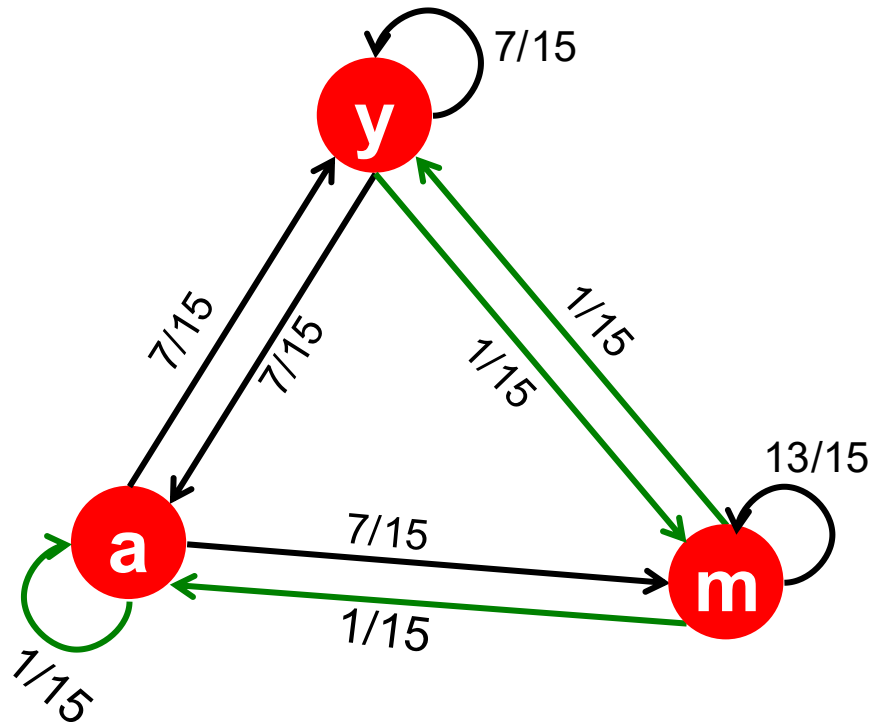
- **We have a recursive problem: $\mathbf{r} = A \cdot \mathbf{r}$**

And the Power method still works!

- **What is β ?**

- In practice $\beta = 0.8, 0.9$ (jump 5 steps on average)

Random Teleports ($\beta = 0.8$)



$$\begin{matrix} & \mathbf{M} & & \mathbf{[1/N]_{N \times N}} \\ 0.8 & \begin{bmatrix} 1/2 & 1/2 & 0 \\ 1/2 & 0 & 0 \\ 0 & 1/2 & 1 \end{bmatrix} & + & 0.2 \begin{bmatrix} 1/3 & 1/3 & 1/3 \\ 1/3 & 1/3 & 1/3 \\ 1/3 & 1/3 & 1/3 \end{bmatrix} \\ & & & \mathbf{A} \\ & \begin{matrix} y & \begin{bmatrix} 7/15 & 7/15 & 1/15 \\ 7/15 & 1/15 & 1/15 \\ 1/15 & 7/15 & 13/15 \end{bmatrix} \\ a \\ m \end{matrix} & & \end{matrix}$$

y	=	1/3	0.33	0.24	0.26	7/33
a		1/3	0.20	0.20	0.18	5/33
m		1/3	0.46	0.52	0.56	21/33

**How do we actually
compute the PageRank?**

Computing PageRank

- **Key step is matrix-vector multiplication**

- $\mathbf{r}^{\text{new}} = \mathbf{A} \cdot \mathbf{r}^{\text{old}}$

- Easy if we have enough main memory to hold $\mathbf{A}$, $\mathbf{r}^{\text{old}}$, $\mathbf{r}^{\text{new}}$

- **Say $N = 1$ billion pages**

- We need 4 bytes for each entry (say)
 - 2 billion entries for vectors, approximately 8GB
 - **Matrix $\mathbf{A}$ has N^2 entries**
 - 10^{18} is a large number!

$$\mathbf{A} = \beta \cdot \mathbf{M} + (1-\beta) [\mathbf{1}/N]_{N \times N}$$

$$\begin{aligned} \mathbf{A} &= 0.8 \begin{bmatrix} \frac{1}{2} & \frac{1}{2} & 0 \\ \frac{1}{2} & 0 & 0 \\ 0 & \frac{1}{2} & 1 \end{bmatrix} + 0.2 \begin{bmatrix} \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \end{bmatrix} \\ &= \begin{bmatrix} \frac{7}{15} & \frac{7}{15} & \frac{1}{15} \\ \frac{7}{15} & \frac{1}{15} & \frac{1}{15} \\ \frac{1}{15} & \frac{7}{15} & \frac{13}{15} \end{bmatrix} \end{aligned}$$

Matrix Formulation

- Suppose there are N pages
- Consider page i , with d_i out-links
- We have $M_{ji} = 1 / |d_i|$ when $i \rightarrow j$ and $M_{ji} = 0$ otherwise
- **The random teleport is equivalent to:**
 - Adding a **teleport link** from i to every other page and setting transition probability to $(1-\beta)/N$
 - Reducing the probability of following each out-link from $1 / |d_i|$ to $\beta / |d_i|$
 - **Equivalent:** Tax each page a fraction $(1-\beta)$ of its score and redistribute evenly

Rearranging the Equation

- $\mathbf{r} = \mathbf{A} \cdot \mathbf{r}$, where $A_{ji} = \beta M_{ji} + \frac{1-\beta}{N}$
- $r_j = \sum_{i=1}^N A_{ji} \cdot r_i$
- $r_j = \sum_{i=1}^N \left[\beta M_{ji} + \frac{1-\beta}{N} \right] \cdot r_i$
 $= \sum_{i=1}^N \beta M_{ji} \cdot r_i + \frac{1-\beta}{N} \sum_{i=1}^N r_i$
 $= \sum_{i=1}^N \beta M_{ji} \cdot r_i + \frac{1-\beta}{N}$ since $\sum r_i = 1$

- So we get: $\mathbf{r} = \beta \mathbf{M} \cdot \mathbf{r} + \left[\frac{1-\beta}{N} \right]_N$

Note: Here we assumed $\mathbf{M}$
has no dead-ends

$[x]_N$... a vector of length N with all entries x

Sparse Matrix Formulation

- We just rearranged the **PageRank equation**

$$\mathbf{r} = \beta \mathbf{M} \cdot \mathbf{r} + \left[\frac{1 - \beta}{N} \right]_N$$

- where $[(1-\beta)/N]_N$ is a vector with all N entries $(1-\beta)/N$
- $\mathbf{M}$ is a **sparse matrix!** (with no dead-ends)
 - 10 links per node, approximately $10N$ entries
- So in each iteration, we need to:
 - Compute $\mathbf{r}^{\text{new}} = \beta \mathbf{M} \cdot \mathbf{r}^{\text{old}}$
 - Add a constant value $(1-\beta)/N$ to each entry in $\mathbf{r}^{\text{new}}$
 - **Note if $\mathbf{M}$ contains dead-ends then $\sum_j r_j^{\text{new}} < 1$ and we also have to renormalize $\mathbf{r}^{\text{new}}$ so that it sums to 1**

PageRank: The Complete Algorithm

- Input: Graph G and parameter β

- Directed graph G (can have **spider traps** and **dead ends**)
- Parameter β

- Output: PageRank vector r^{new}

- **Set:** $r_j^{old} = \frac{1}{N}$
- **repeat until convergence:** $\sum_j |r_j^{new} - r_j^{old}| > \epsilon$
 - $\forall j: r_j'^{new} = \sum_{i \rightarrow j} \beta \frac{r_i^{old}}{d_i}$
 $r_j'^{new} = 0$ if in-degree of j is 0
 - **Now re-insert the leaked PageRank:**
 $\forall j: r_j^{new} = r_j'^{new} + \frac{1-S}{N}$ **where:** $S = \sum_j r_j'^{new}$
 - $r^{old} = r^{new}$

If the graph has no dead-ends then the amount of leaked PageRank is $1-\beta$.
But since we have dead-ends the amount of leaked PageRank may be larger.
We have to explicitly account for it by computing S .

Some Problems with PageRank

- **Measures generic popularity of a page**
 - Biased against topic-specific authorities
 - **Solution:** Topic-Specific PageRank
- **Uses a single measure of importance**
 - Other models of importance
 - **Solution:** Hubs-and-Authorities
- **Susceptible to Link spam**
 - Artificial link topographies created in order to boost page rank
 - **Solution:** TrustRank